

TRƯỜNG ĐẠI HỌC KHOA HỌC HUẾ

KHOA CÔNG NGHỆ THÔNG TIN



TIỂU LUẬN

PHÁT TRIỂN ỨNG DỤNG IOT

ĐỀ TÀI: Giám sát năng lượng tiêu thụ với ESP32

Giảng viên hướng dẫn: Võ Việt Dũng

Sinh viên thực hiện: Nguyễn An Phú

Nhóm học phần: 2025-2026.2.TIN4024.003

Mã sinh viên: 22T1020806

Lớp: K46 – K

DANH MỤC CÁC TỪ VIẾT TẮT

IoT: Internet of Things - Internet vạn vật

SD: Secure Digital – bảo mật kỹ thuật số

SoC: System On a Chip - Hệ thống trên một vi mạch

GHz: Gigahertz

AC: Alternating Current – dòng điện xoay chiều

DC: Direct Current – dòng điện một chiều

TSMC: Taiwan Semiconductor Manufacturing Co., Ltd - Công ty TNHH Sản xuất Chất bán dẫn Đài Loan

ADC: (trong ESP32) Analog-to-Digital Converter

MỤC LỤC

	Trang
MỞ ĐẦU	4
NỘI DUNG	5
CHƯƠNG I. CƠ SỞ LÝ THUYẾT	5
I- Các khái niệm liên quan:	5
II- Nguyên lý đo lường các thông số điện năng:	6
II.1. Nguyên lý đo dòng điện (Hiệu ứng Hall)	6
II.2. Nguyên lý đo điện áp (Phân áp điện trở)	6
II.3. Nguyên lý tính toán công suất và điện năng	7
CHƯƠNG II. GIẢI PHÁP HỆ THỐNG	8
I- Sơ đồ khối biểu thị cách hệ thống sẽ hoạt động:	8
II- Nền tảng gửi thông tin Blynk:	9
CHƯƠNG III. THIẾT KẾ VÀ MÔ PHỎNG	10
I- Thiết kế mạch ESP32 trên Wokwi:	10
II- Giao diện Blynk:	11
KẾT LUẬN	14
TÀI LIỆU THAM KHẢO	15
PHỤ LỤC	16

MỞ ĐẦU

Trong bối cảnh cuộc Cách mạng công nghiệp 4.0 đang diễn ra mạnh mẽ, việc ứng dụng công nghệ để quản lý và tối ưu hóa việc sử dụng năng lượng ngày càng trở nên quan trọng đối với cả hộ gia đình lẫn các tổ chức, doanh nghiệp. Việc theo dõi, kiểm soát lượng điện tiêu thụ một cách chính xác, kịp thời không chỉ giúp tiết kiệm chi phí mà còn góp phần sử dụng năng lượng hiệu quả và bền vững. Cùng với đó, sự phát triển nhanh chóng của Internet vạn vật (IoT) đã mở ra nhiều giải pháp giám sát thông minh, cho phép người dùng thu thập và theo dõi dữ liệu từ xa theo thời gian thực.

Xuất phát từ thực tế đó, bài tiểu luận sẽ được lựa chọn nghiên cứu đề tài “Giám sát năng lượng tiêu thụ với ESP32”. Đề tài này tập trung xây dựng hệ thống giám sát điện năng dựa trên vi điều khiển ESP32, kết hợp với cảm biến dòng điện ACS712 nhằm đo lường các thông số điện năng và truyền dữ liệu lên nền tảng đám mây để theo dõi và quản lý.

Mục tiêu của bài tiểu luận này là thiết kế sơ đồ nguyên lý của hệ thống, xây dựng mô hình mô phỏng trên nền tảng Wokwi, đồng thời đánh giá khả năng ứng dụng của giải pháp trong thực tế. Qua đó, đề tài góp phần minh họa tiềm năng của các hệ thống IoT trong việc hỗ trợ quản lý năng lượng thông minh và nâng cao hiệu quả sử dụng điện trong đời sống.

Mục tiêu nghiên cứu:

- Xây dựng hệ thống giám sát năng lượng tiêu thụ sử dụng vi điều khiển ESP32.
- Đo lường dòng điện tiêu thụ bằng cảm biến ACS712.
- Hiển thị và theo dõi dữ liệu thông qua nền tảng mô phỏng.

Phạm vi nghiên cứu:

- Nghiên cứu thiết kế và mô phỏng hệ thống trên nền tảng Wokwi.
- Tập trung vào việc đo và giám sát dòng điện tiêu thụ cơ bản, chưa triển khai trên hệ thống thực tế quy mô lớn.

NỘI DUNG

CHƯƠNG I. CƠ SỞ LÝ THUYẾT

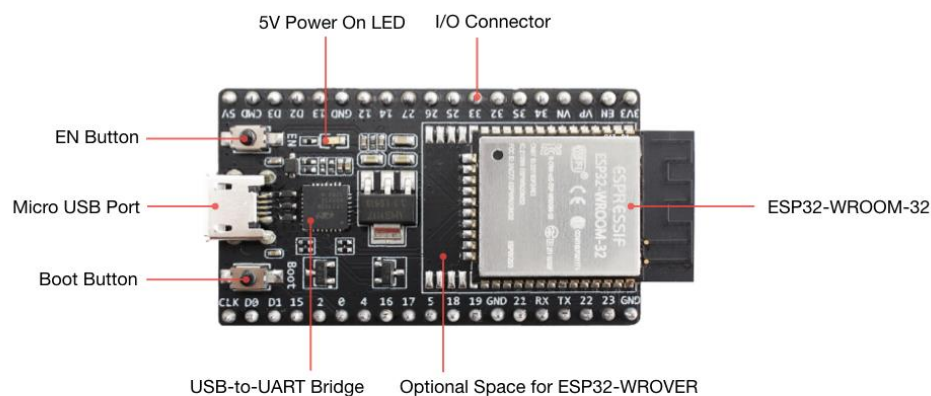
I- Các khái niệm liên quan:

ESP32 là một vi xử lý tích hợp Wi-Fi và Bluetooth tần số 2,4 GHz, được thiết kế dựa trên công nghệ 40 nm tiết kiệm năng lượng của TSMC. [3]



Cảm biến dòng điện ACS712. Nguồn: <https://www.allegromicro.com/-/media/images/products/0712/acs712-product-image.jpg>

ACS712: “Allegro® ACS712 cung cấp các giải pháp tiết kiệm chi phí và chính xác cho việc đo dòng điện xoay chiều (AC) hoặc một chiều (DC) trong các hệ thống công nghiệp, ô tô, thương mại và viễn thông.” [4]



Vi xử lý ESP32. Nguồn: https://docs.espressif.com/projects/esp-idf/en/v5.1/esp32/_images/esp32-devkitc-functional-overview.jpg

Hiệu ứng Hall là hiện tượng trong đó một điện trường ngang được hình thành trong vật liệu rắn khi vật liệu dẫn điện được đặt trong một từ trường vuông góc với dòng điện,... được phát hiện vào năm 1879 bởi Edwin Herbert Hall [6]. Đây là nguyên lý mà ACS712 hoạt động dựa vào.

II- Nguyên lý đo lường các thông số điện năng:

II.1. Nguyên lý đo dòng điện (Hiệu ứng Hall)

Điện áp Hall, được ký hiệu là V_H , được tính theo công thức:

$$V_H = \frac{IB}{qnd}$$

Trong đó:

- V_H là điện áp Hall. Đơn vị: V – Volt.
- I là cường độ dòng điện chạy qua cảm biến. Đơn vị: A – Ampe
- B là cường độ từ trường. Đơn vị: T – Tesla
- q là điện tích. Đơn vị: C - Coulomb
- n là số lượng hạt mang điện trên đơn vị thể tích. Đơn vị: m^{-3}
- d là độ dày của cảm biến. Đơn vị: m – mét

II.2. Nguyên lý đo điện áp (Phân áp điện trở)

Vì các chân Analog - ADC của ESP32 chỉ có thể đọc được điện áp trong khoảng 0V - 3.3V, nên để đo các nguồn điện lớn hơn (như pin 12V hoặc điện lưới sau khi hạ áp), ta cần sử dụng mạch chia điện áp (Voltage Divider).

Mạch chia điện áp là một mạch rất phổ biến, có chức năng lấy điện áp cao hơn và chuyển đổi nó thành điện áp thấp hơn bằng cách sử dụng một cặp điện trở. Công thức tính điện áp đầu ra dựa trên định luật Ohm [5] và có công thức:

$$V_{out} = \frac{V_s \times R_2}{(R_1 + R_2)}$$

Trong đó:

- V_s là điện áp nguồn, tính bằng volt (V),
- R_1 là điện trở của điện trở thứ nhất, tính bằng ohm (Ω).

- R_2 là điện trở của điện trở thứ hai, tính bằng ohm (Ω).
- V_{out} là điện áp đầu ra và đưa vào chân ADC của ESP32, tính bằng volt (V), [5]

II.3. Nguyên lý tính toán công suất và điện năng

Dòng điện cho vi xử lý ESP32 là dòng điện một chiều. Do đó, ta sẽ tính công suất và điện năng như sau:

Công suất tức thời của dòng điện: được xác định bởi tích của điện áp và cường độ dòng điện: [1]:

$$P = U * I \text{ (W)}$$

Trong đó:

- P : Công suất tức thời của dòng điện tại thời điểm đo. Đơn vị: W – Watt
- U : hiệu điện thế . Đơn vị: V – Volt
- I : cường độ dòng điện. Đơn vị: A -Ampe [1]

Điện năng tiêu thụ

Công thức tính điện năng:

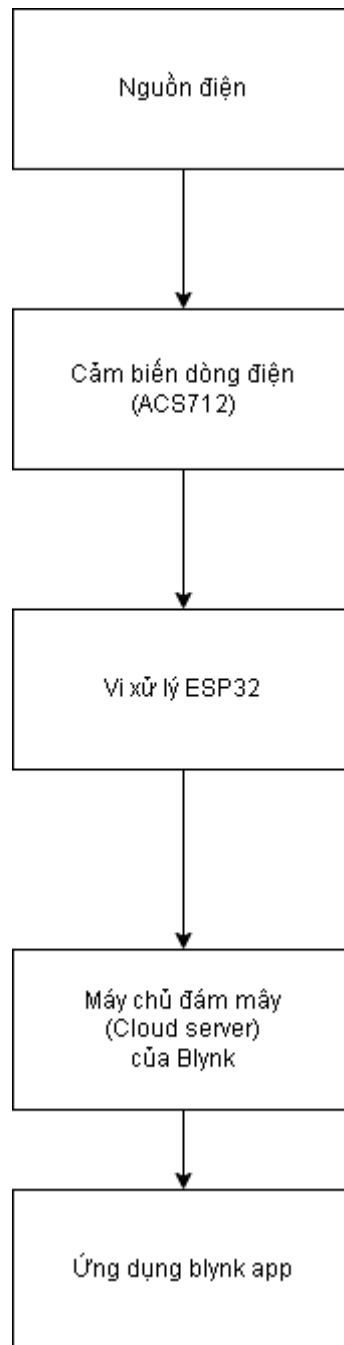
$$A = P.t = U.I.t$$

Trong đó:

- A : Điện năng (J –Joule).
- P : Công suất (W - Watt).
- U : Điện áp (V - Volt).
- I : Cường độ dòng điện (A - Ampe).
- t : Thời gian (s - giây). [1]

CHƯƠNG II. GIẢI PHÁP HỆ THỐNG

I- Sơ đồ khối biểu thị cách hệ thống sẽ hoạt động:



Hệ thống hoạt động theo một quy trình khép kín từ việc thu thập dữ liệu vật lý đến việc hiển thị số hóa, cụ thể qua 4 giai đoạn:

- **Giai đoạn thu thập (Data Acquisition):** Dòng điện từ **Nguồn** đi qua tải sẽ được **Cảm biến ACS712** giám sát. Dựa trên hiệu ứng Hall, cảm biến chuyển đổi cường độ dòng điện thành mức điện áp analog tương ứng.

- **Giai đoạn xử lý (Data Processing):** ESP32 tiếp nhận tín hiệu analog thông qua bộ chuyển đổi ADC (Analog-to-Digital Converter). Tại đây, vi xử lý thực hiện các phép tính toán dựa trên các công thức lý thuyết (như tính giá trị RMS, công suất tức thời $P=U \times I$ và tích phân điện năng tiêu thụ theo thời gian).
- **Giai đoạn truyền tải (Communication):** Sau khi có dữ liệu số, ESP32 sử dụng module Wi-Fi tích hợp để đóng gói dữ liệu và gửi lên **Blynk Cloud** thông qua giao thức truyền tin bảo mật.
- **Giai đoạn hiển thị (Visualization):** **Blynk App** trên điện thoại truy vấn dữ liệu từ Cloud để cập nhật lên giao diện người dùng, giúp giám sát mọi lúc mọi nơi.

II- Nền tảng gửi thông tin Blynk:

Blynk là một nền tảng phần mềm IoT hoàn chỉnh, nơi “bạn có thể tạo nguyên mẫu, triển khai và quản lý từ xa các thiết bị điện tử được kết nối ở mọi quy mô.” [\[7\]](#) .

Với Blynk, người dùng có thể tạo ra các ứng dụng điều khiển thiết bị IoT chỉ trong vài phút mà không cần có kiến thức chuyên sâu về lập trình. Blynk cung cấp một loạt các công cụ và giao diện người dùng trực quan giúp người dùng dễ dàng kéo thả và tương tác với các thành phần điều khiển, biểu đồ, cảm biến, và nhiều tính năng khác.[\[2\]](#)

Blynk không chỉ đơn thuần là một ứng dụng hiển thị, mà là một hệ sinh thái IoT gồm 3 thành phần cốt lõi tương tác chặt chẽ:

- **Blynk App:** Cho phép thiết kế giao diện (Dashboard) bằng cách kéo-thả các Widget (biểu đồ, nút nhấn, màn hình hiển thị). Trong đề tài này, các Widget được cấu hình để hiển thị các thông số U , I , P , E .
- **Blynk Server (Cloud):** Đóng vai trò là "bộ não" trung gian, chịu trách nhiệm điều phối toàn bộ lưu lượng tin nhắn giữa phần cứng (ESP32) và phần mềm (App). Nó lưu trữ lịch sử dữ liệu tiêu thụ để người dùng có thể xem lại theo ngày/tháng/năm.
- **Blynk Libraries:** Các thư viện mã nguồn mở được cài đặt trên ESP32 để đơn giản hóa việc kết nối và giao tiếp. Một khái niệm quan trọng là **Virtual Pins (Chân ảo)**, cho phép gửi bất kỳ loại dữ liệu nào (số thực, chuỗi) mà không bị giới hạn bởi các chân vật lý của vi điều khiển

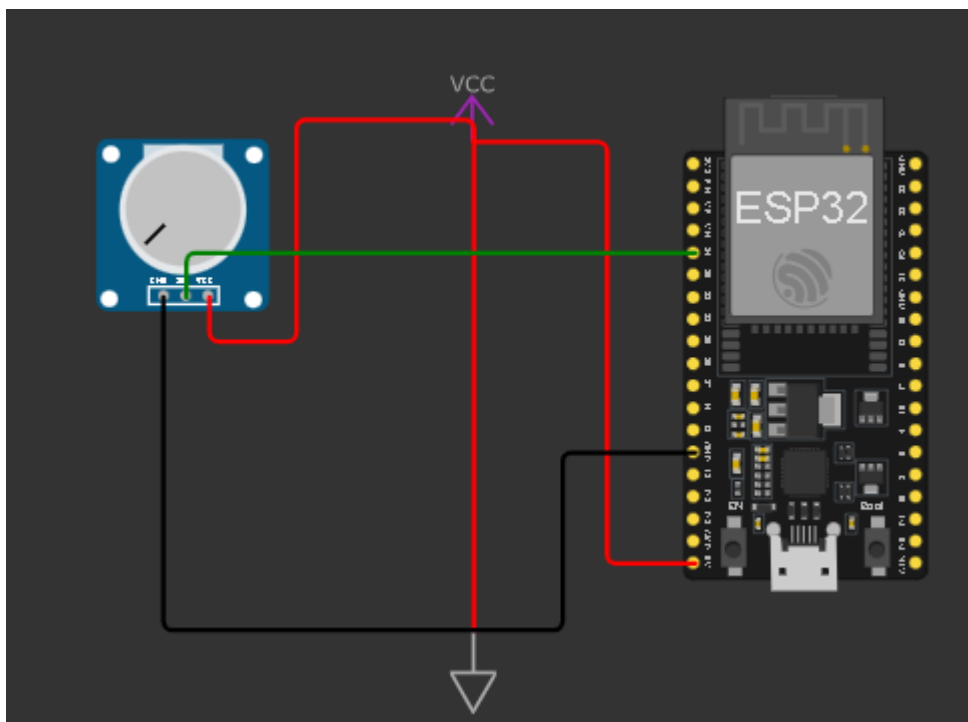
CHƯƠNG III. THIẾT KẾ VÀ MÔ PHỎNG

I- Thiết kế mạch ESP32 trên Wokwi:

Bảng các linh kiện sử dụng trong mô phỏng wokwi:

STT	Tên linh kiện	Số lượng
1	ESP32 DevKit V4	1
2	Wokwi Potentiometer	1

Bảng mạch:



Mạch ESP32 thiết kế trên Wokwi.com. Nguồn:

<https://wokwi.com/projects/387826007234326529>

Trong môi trường mô phỏng Wokwi, do hạn chế về việc chưa có sẵn module ACS712 thực tế, hệ thống đã sử dụng linh kiện thay thế tương đương để kiểm chứng thuật toán xử lý dữ liệu:

- **Thành phần chính:** Sử dụng bo mạch ESP32 DevKit V1 làm trung tâm điều khiển.
- **Cơ chế giả lập dòng điện:** Sử dụng một **Biến trở (Slide Potentiometer)** để giả lập tín hiệu điện áp từ cảm biến ACS712. Khi kéo thanh trượt, mức điện áp thay

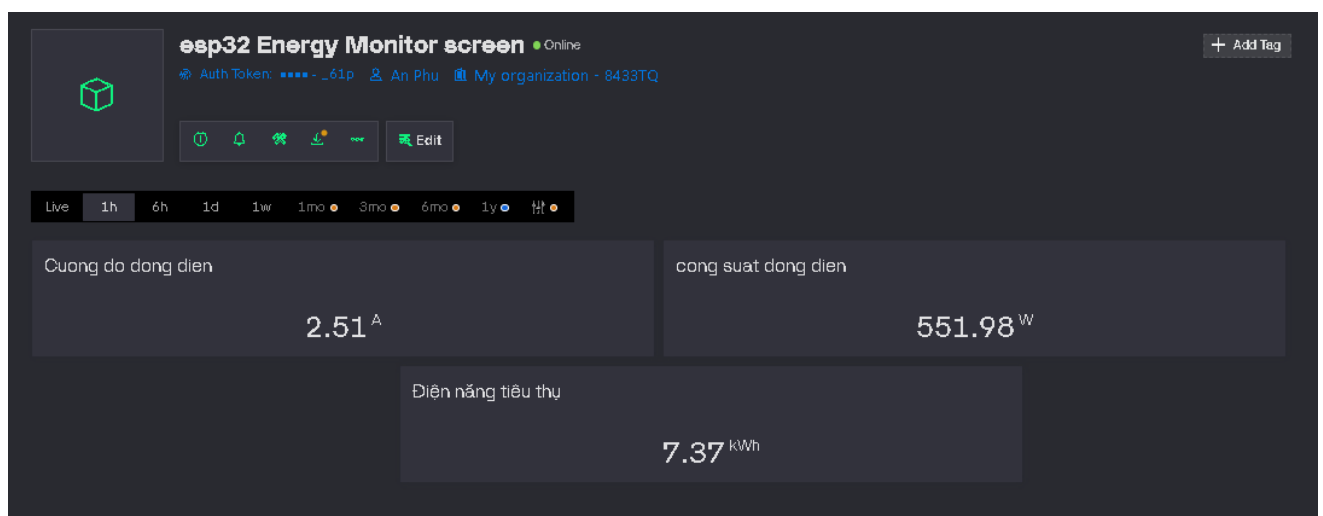
đổi (từ 0V đến 3.3V) tương ứng với sự thay đổi của dòng điện tiêu thụ trong thực tế.

- **Sơ đồ kết nối:**

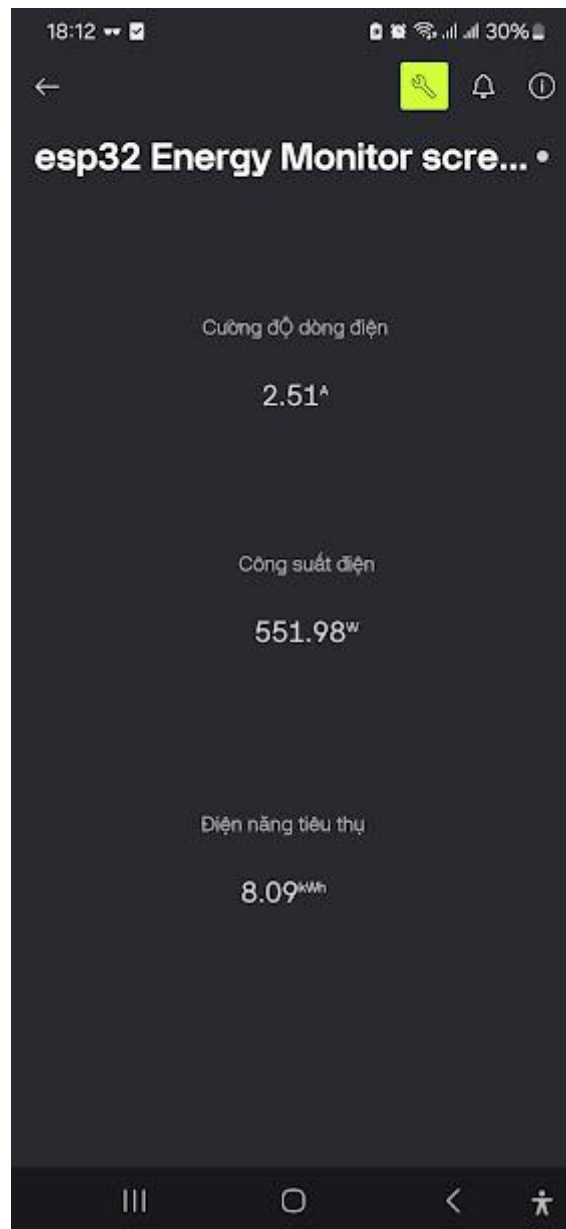
- Chân tín hiệu của biến trở được nối vào chân **GPIO 34** của ESP32 (đây là chân ADC1 có độ phân giải cao).
- Các chân nguồn VCC và GND được nối chung vào đường nguồn 3.3V của ESP32 để đảm bảo mức logic an toàn.

- **Mục tiêu mô phỏng:** Kiểm tra khả năng đọc tín hiệu analog ổn định của ESP32 và độ chính xác của logic chuyển đổi từ giá trị điện áp sang thông số công suất hiển thị.

II- Giao diện Blynk:



Giao diện trên pc tại blynk.cloud



Giao diện trên điện thoại tại ứng dụng “Blynk IoT”

Giao diện trên Blynk App được thiết kế hướng tới trải nghiệm người dùng trực quan với các thành phần:

- Gauge Widget (Đồng hồ kim): Hiển thị Công suất (W) thời gian thực, giúp người dùng nhận biết ngay lập tức nếu có thiết bị điện nào đang tiêu thụ quá mức.
- Value Display: Hiển thị chi tiết các chỉ số Điện áp (V) và Cường độ dòng điện (A).

- SuperChart: Đây là thành phần quan trọng nhất để theo dõi xu hướng. Biểu đồ này lưu trữ lịch sử tiêu thụ điện năng (kWh), cho phép so sánh mức sử dụng điện giữa các khung giờ trong ngày để đưa ra kế hoạch tiết kiệm điện hiệu quả.

KẾT LUẬN

Thông qua việc nghiên cứu và thiết kế hệ thống giám sát năng lượng tiêu thụ sử dụng ESP32, tiểu luận đã hoàn thành các mục tiêu đề ra:

- Xây dựng được cơ sở lý thuyết về đo lường điện năng và nguyên lý hoạt động của cảm biến ACS712.
- Thiết kế thành công sơ đồ nguyên lý và mô phỏng trên nền tảng Wokwi, đảm bảo tính kết nối giữa các linh kiện điện tử.
- Đề xuất giải pháp truyền dẫn và hiển thị dữ liệu trực quan thông qua nền tảng Blynk.

Mặc dù việc mô phỏng trên Wokwi có những hạn chế về sai số thực tế của linh kiện, nhưng mô hình này đã cung cấp một cái nhìn tổng quan và nền tảng lý thuyết vững chắc cho việc triển khai thực tế. Trong tương lai, hệ thống có thể phát triển thêm tính năng cảnh báo khi quá tải hoặc tích hợp các thuật toán dự báo tiêu thụ điện năng dựa trên dữ liệu lịch sử.

TÀI LIỆU THAM KHẢO

Tiếng Việt:

[1] www.saigonsinco.com – Sinco Engineering: Dòng điện 1 chiều là gì? Đặc điểm, công thức tính & ứng dụng. <https://www.saigonsinco.com/blogs/kien-thuc-chuyen-nganh/dong-dien-1-chieu-la-gi/> . Truy cập ngày 30/03/2026

[2] Bùi Quang Dũng. Blynk là gì? Hướng dẫn cài đặt và sử dụng công cụ Blynk 2.0 cực kì đơn giản . <https://fptshop.com.vn/tin-tuc/danh-gia/blynk-la-gi-184300> . Truy cập ngày 30/06/2026

Tiếng Anh:

[3] Espressif Systems. *ESP32 Series Datasheet Version 5.2*. https://documentation.espressif.com/esp32_datasheet_en.pdf . Trang 2. Truy cập ngày 29/03/2026

[4] Allegro MicroSystems: *ACS712 Datasheet*. <https://www.alldatasheet.com/datasheet-pdf/view/168326/ALLEGRO/ACS712.html> . Trang 1. Truy cập ngày 29/03/2026

[5] ohmslawcalculator.com: *Voltage Divider Calculator*. <https://ohmslawcalculator.com/voltage-divider-calculator/> . Truy cập ngày 30/03/2026

[6] byjus.com : *Principle of Hall effect* . <https://byjus.com/physics/hall-effect/> . Truy cập vào ngày 30/03/2026

[7] Blynk.io: *Introduction*. <https://docs.blynk.io/en/> . Truy cập vào ngày 30/03/2026

PHỤ LỤC

→ Các dòng code trong dự án mô phỏng wokwi

- diagram.json:

```
{  
  "version": 1,  
  "author": "Anonymous maker",  
  "editor": "wokwi",  
  "parts": [  
    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 0, "left": 91.24, "attrs": {} },  
    { "type": "wokwi-potentiometer", "id": "pot1", "top": 17.9, "left": -163.4, "attrs": {}  
  },  
    { "type": "wokwi-vcc", "id": "vcc1", "top": -8.84, "left": -9.6, "attrs": {} },  
    { "type": "wokwi-gnd", "id": "gnd1", "top": 230.4, "left": -10.2, "attrs": {} }  
  ],  
  "connections": [  
    [ "esp:TX", "$serialMonitor:RX", "", [] ],  
    [ "esp:RX", "$serialMonitor:TX", "", [] ],  
    [ "vcc1:VCC", "gnd1:GND", "red", [ "v0" ] ],  
    [ "pot1:VCC", "vcc1:VCC", "red", [ "v19.2", "h37.6", "v-96", "h9.6" ] ],  
    [ "pot1:GND", "gnd1:GND", "black", [ "v0" ] ],  
    [ "esp:5V", "vcc1:VCC", "red", [ "h-38.25", "v-182.4" ] ],  
    [ "esp:GND.1", "gnd1:GND", "black", [ "h-57.45", "v76.8" ] ],  
    [ "pot1:SIG", "esp:34", "green", [ "v0" ] ]  
  ]  
}
```



```
],  
  
"dependencies": {}  
  
}
```

- main.cpp:

```
#define BLYNK_AUTH_TOKEN "cqkq1WqQWrVPce88BJXFTGxIXaen_61p"
```

```
#define BLYNK_TEMPLATE_ID "TMPL6mWQWvg2r"
```

```
#define BLYNK_TEMPLATE_NAME "esp32 Energy Monitor"
```

```
// Definitions above may differ
```

```
#define BLYNK_PRINT Serial
```

```
#include <WiFi.h>
```

```
#include <BlynkSimpleEsp32.h>
```

```
#include "ACS712.h"
```

```
char auth[] = BLYNK_AUTH_TOKEN;
```

```
char ssid[] = "Wokwi-GUEST";
```

```
char pass[] = "";
```

```
ACS712 ACS(34, 3.3, 4095, 185);
```

```
float energy = 0;
```

```
unsigned long lastTime = 0;
```

```

void setup() {
    Serial.begin(115200);

    Blynk.begin(auth, ssid, pass);

    ACS.autoMidPoint();

    lastTime = millis();
}

void loop() {
    Blynk.run();

    int mA = ACS.mA_AC_sampling();

    float current = mA / 1000.0;

    float voltage = 220.0;

    float power = voltage * current;

    unsigned long now = millis();

    float dt = (now - lastTime) / 1000.0;

    lastTime = now;

    energy += power * (dt / 3600.0);

    // Gửi lên Blynk

```

```
Blynk.virtualWrite(V0, current);  
  
Blynk.virtualWrite(V1, power);  
  
Blynk.virtualWrite(V2, energy);  
  
  
Serial.println(power);  
  
  
delay(1000);  
}
```